FinSight — AWS Viva Guide

Prepared for **Aakar Gupta (24BRS1321), VIT Chennai** · Project: **FinSight — Cloud-Based Personal Finance Intelligence & Expense Analytics Platform**.

This guide combines the AWS syllabus cheat sheet (Part I: eight modules, definitions, comparisons and one-line viva answers) with the project-specific viva (Part II: 150 questions, each answer directly below its question). Answers describe the **actual** implementation and the **live** deployment (stack `finance-intelligence-prod`, ap-south-1, verified 2026-10-16): API Gateway + Lambda hosting, DynamoDB, private S3, Cognito, CloudWatch, ECR, IAM and SAM. **CloudFront is in the template but not deployed** (account verification pending). Technical details: PROJECT_DOCUMENTATION.md.

Contents

- Part I — AWS syllabus cheat sheet
- Module 1 — AWS Cloud Foundations and IAM
- Module 2 — Compute and Storage
- Module 3 — Networking and Content Delivery
- Module 4 — Database Services
- Module 5 — Management and Cost
- Module 6 — Security
- Module 7 — Integration and Architecture
- Module 8 — Migration and Hybrid Architecture
- Quick comparisons (frequent viva pairs)
- Part II — FinSight project viva (150 questions)
- A. Project overview
- B. Problem statement and objectives
- C. Frontend architecture
- D. Backend architecture
- E. API design
- F. Authentication and authorization
- G. DynamoDB data model
- H. S3 receipt storage
- I. OCR and Tesseract
- J. AWS Lambda
- K. API Gateway
- L. Amazon Cognito
- M. CloudWatch
- N. IAM
- O. SAM and CloudFormation
- P. Deployment process
- Q. Security
- R. Testing strategy

- S. Cost optimisation
- T. CloudFront limitation
- U. Token revocation limitation
- V. Alternative architecture choices
- W. Database comparisons
- X. Serverless versus EC2
- Y. S3 versus EBS versus EFS
- Z. DynamoDB versus RDS (short)
- AA. High availability and scalability
- AB. Difficult project-specific questions
- AC. Rapid-fire
- AD. Mock viva (short answers)

Part I — AWS syllabus cheat sheet

Prepared for: **Aakar Gupta (24BRS1321), VIT Chennai** · Project: FinSight. Legend: **[FinSight]** = used in the deployed FinSight stack (ap-south-1). **[Planned]** = in the FinSight template but not deployed. **[Syllabus]** = studied for the course, not used in FinSight. Facts about AWS service status reflect AWS documentation as of late 2026. Check the AWS console or docs for anything that changes often (prices, device availability).

Each topic follows the same pattern: **Def** (definition) · **Why** (purpose) · **Parts** (components) · **+** (advantages) · **–** (limitations) · **Eg** (example) · **Viva** (a one-line answer) · **vs** (comparison).

Module 1 — AWS Cloud Foundations and IAM

Cloud computing

- **Def:** on-demand delivery of compute, storage, databases and other IT resources over the internet with pay-as-you-go pricing.
- **Why:** avoid buying and running data centres; scale with demand.
- **Parts:**
- service models: IaaS (EC2), PaaS (Elastic Beanstalk), SaaS (WorkMail), and serverless/FaaS (Lambda);
- deployment models: public, private, hybrid.
- **+** No upfront capital; elasticity; global reach in minutes; you pay for what you use.
- **–** Ongoing operating cost; dependence on the provider; shared-responsibility work stays with you; egress cost.
- **Eg:** FinSight runs with no servers to manage: Lambda, API Gateway, DynamoDB.
- **Viva:** "Cloud computing is renting IT capacity on demand over the internet and paying only for what you use."
- **vs:** On-premises means CapEx, fixed capacity and full responsibility.

AWS global infrastructure

- **Def:** Regions (geographic areas), each with ≥ 3 isolated Availability Zones (one or more data centres each), plus edge locations for CloudFront and Route 53.
- **Why:** low latency, data residency, fault isolation.

- **Parts:**
- Regions (e.g. ap-south-1 Mumbai);
- AZs (ap-south-1a/b/c);
- edge locations and regional edge caches;
- Local Zones and Wavelength Zones.
- **+** Multi-AZ designs survive a data-centre failure; you choose where data lives.
- **–** Not every service or feature exists in every Region; cross-Region replication is your design work.
- **Eg [FinSight]:** all resources are in ap-south-1. Lambda, DynamoDB, S3 and API Gateway are Regional services that span AZs automatically.
- **Viva:** "A Region is a geographic area with multiple isolated AZs; edge locations cache content close to users."

Shared responsibility model

- **Def:** AWS secures the *cloud* (hardware, facilities, network, managed-service software); the customer secures what is *in* the cloud (data, IAM, configuration, code, encryption choices).
- **Why:** clear security ownership.
- **The line moves with the service.** On EC2 you patch the OS; on Lambda AWS patches the runtime and you own the code and IAM.
- **Eg [FinSight]:** AWS runs the Lambda runtime and DynamoDB storage. FinSight owns the IAM least privilege, the S3 public-access block, token validation and input validation.
- **Viva:** "AWS is responsible *of* the cloud, the customer *in* the cloud; the more managed the service, the less the customer manages."

IAM — users, groups, roles, policies

- **Def:** Identity and Access Management is the global service that controls *who* (authentication) can do *what* on *which* resources (authorization).
- **Parts:**
- **users** (long-term identities);
- **groups** (collections of users);
- **roles** (assumed identities with temporary credentials — for services, federation and cross-account access);
- **policies** (JSON documents with Effect/Action/Resource/Condition): identity-based, resource-based (e.g. an S3 bucket policy), permission boundaries, SCPs;
- MFA, access keys, IAM Identity Center.
- **Evaluation:** an explicit **Deny** beats an Allow; with no Allow the default is deny.
- **+** Fine-grained, free, auditable (CloudTrail).
- **–** Policy sprawl; mistakes such as wildcards are dangerous.
- **Eg [FinSight]:** each Lambda has its own role (`AWSLambdaBasicExecutionRole` + one table policy; only the receipts function gets S3 Get/Put/Delete). The receipts bucket policy denies non-TLS access.
- **Viva:** "Users and groups are for people, roles give temporary credentials to services or federated users, and policies define the permissions."
- **vs:** A role has no long-term password or keys; a user does.

Least privilege

- **Def:** grant only the permissions a task needs, on the specific resources it needs.
- **How:** scope actions and resources; use conditions; review with IAM Access Analyzer and last-accessed data; prefer roles over keys.
- **Eg [FinSight]:** the table policy lists 8 item-level actions on one table and its indexes, with no `Scan` and no `dynamodb:*`.
- **Viva:** "Start from nothing and add only what is required."

Authentication vs authorization

- **Authentication** = proving who you are (password + MFA, Cognito sign-in, SigV4 signatures). **Authorization** = deciding what you may do (IAM policies, a JWT authorizer, application checks).
- **Eg [FinSight]:** Cognito authenticates the user; the API Gateway JWT authorizer and the backend authorize every request and scope data by the token's `sub`.
- **Viva:** "AuthN is identity; AuthZ is permission."

Module 2 — Compute and Storage

Amazon EC2 [Syllabus]

- **Def:** resizable virtual servers (instances) in the cloud.
- **Parts:**
- AMI;
- instance type;
- EBS volumes or instance store;
- security groups;
- key pairs;
- Elastic IP;
- user data;
- placement groups.
- **+** Full OS control; any software; many sizes.
- **−** You patch, scale and secure the OS; you pay while it runs, even when idle.
- **Eg:** a web server on Amazon Linux behind an ALB.
- **Viva:** "EC2 is IaaS: virtual machines you control and pay for by the second or hour."
- **vs Lambda:** EC2 is always-on and OS-managed by you. Lambda is event-driven, has no servers to manage, is billed per request and per millisecond, and has a 15-minute limit. FinSight chose Lambda for its idle cost of ~0.

EC2 instance types and pricing

- **Families:**
- general purpose (t, m);
- compute optimised (c);
- memory optimised (r, x);
- storage optimised (i, d);
- accelerated (p, g, inf).

- **Naming:** `m7g.large` = family m, generation 7, `g` = Graviton (Arm), size large.
- **Pricing:**
- **On-Demand** (no commitment);
- **Savings Plans** / **Reserved Instances** (1–3-year commitment, up to ~72% off);
- **Spot** (spare capacity, up to ~90% off, can be interrupted with a 2-minute notice);
- **Dedicated Hosts/Instances** (compliance and licensing).
- **Viva:** "On-Demand for flexibility, Savings Plans/RIs for steady load, Spot for fault-tolerant batch work."

EC2 instance lifecycle

- **States:** pending → running → stopping → stopped (EBS-backed only) → (start again) / shutting-down → terminated; also hibernate.
- Stopped instances incur no compute charge, but EBS storage is still billed. A stop/start may move the instance to new hardware (the public IP changes unless it is an Elastic IP).
- **Viva:** "Stop keeps the EBS root volume and data; terminate deletes the instance (and the root volume by default)."

EC2 Auto Scaling [Syllabus]

- **Def:** automatically adjusts the number of EC2 instances in an Auto Scaling group between min and max.
- **Parts:**
- launch template;
- ASG (min/desired/max, subnets across AZs);
- scaling policies: target tracking (e.g. 50% CPU), step, scheduled, predictive;
- health checks that replace unhealthy instances.
- **+** Elasticity, availability, cost control.
- **–** Scale-out takes minutes; instances must be stateless.
- **Viva:** "Auto Scaling keeps the right number of healthy instances across AZs."
- **vs Lambda:** Lambda scales per request automatically, with no group to manage (FinSight).

Elastic Load Balancing [Syllabus]

- **Def:** distributes traffic across targets in multiple AZs.
- **Types:**
- **ALB** — Layer 7 HTTP/HTTPS; path/host routing; Lambda targets; WAF;
- **NLB** — Layer 4 TCP/UDP; very high performance; static IPs;
- **GWLB** — inline security appliances;
- **CLB** — legacy.
- **+** Health checks, TLS termination, high availability.
- **–** Hourly cost plus LCU charges even when idle.
- **Viva:** "ALB for HTTP routing, NLB for raw TCP/UDP performance."
- **vs API Gateway:** API Gateway adds auth, throttling and per-request pricing, which suits APIs with no idle cost (FinSight).

Amazon S3 [FinSight]

- **Def:** object storage. Buckets hold objects (keys) up to 5 TB each, with 11 nines of durability.
- **Parts:**
- buckets (globally unique names, Regional);
- versioning;
- bucket policies and Block Public Access;
- SSE-S3/SSE-KMS encryption;
- presigned URLs;
- CORS;
- lifecycle rules;
- event notifications;
- replication.
- **+** Virtually unlimited, cheap, durable, serverless.
- **–** Not a file system or a block device; object-level operations only.
- **Eg [FinSight]:** a private receipts bucket: Block Public Access, SSE-S3, a TLS-only policy, browser upload by 5-minute presigned POST (size and type enforced), viewing by 2-minute presigned GET.
- **Viva:** "S3 is durable object storage accessed over HTTPS; access is controlled by IAM, bucket policies and presigned URLs."

S3 storage classes and lifecycle rules

- **Classes:**
- Standard;
- Intelligent-Tiering (automatic tiering);
- Standard-IA;
- One Zone-IA;
- Glacier Instant Retrieval;
- Glacier Flexible Retrieval (minutes to hours);
- Glacier Deep Archive (12–48 h, cheapest);
- Express One Zone (single-digit ms).
- **Lifecycle** rules transition objects between classes or expire them after N days, and can abort incomplete multipart uploads.
- **Eg [FinSight]:** a lifecycle rule aborts incomplete multipart uploads after 1 day. An improvement would expire unconfirmed receipts.
- **Viva:** "Choose the class by access frequency and retrieval time; lifecycle rules automate moves and deletion."

Amazon EBS [Syllabus]

- **Def:** block storage volumes for EC2, in one AZ.
- **Types:** gp3/gp2 (SSD general purpose), io2/io1 (provisioned IOPS), st1 (throughput HDD), sc1 (cold HDD).
- Snapshots go to S3 (incremental) and can be encrypted.
- **+** Low-latency disk for databases and OS volumes.
- **–** AZ-bound; normally one instance attaches (except io2 Multi-Attach).
- **Viva:** "EBS is a network-attached virtual hard disk for one EC2 instance in one AZ."

Amazon S3 Glacier [Syllabus]

- **Def:** archival storage, now offered as **S3 Glacier storage classes** (Instant Retrieval, Flexible Retrieval, Deep Archive). The original Glacier vault API still exists as "S3 Glacier (vaults)".
- **+** Very low cost per GB.
- **–** Retrieval time and fees; minimum storage durations (90/180 days).
- **Eg:** keeping 7-year financial records.
- **Viva:** "Glacier is for data you rarely read; you trade retrieval speed for price."

Amazon EFS [Syllabus]

- **Def:** a managed, elastic NFS file system shared by many EC2 instances, containers or Lambda functions across AZs.
- **+** Shared POSIX file system that grows automatically.
- **–** Higher cost per GB than S3/EBS; NFS (Linux) only.
- **vs:**
- **S3** = objects over HTTPS (cheapest, unlimited);
- **EBS** = block disk for one instance in one AZ;
- **EFS** = shared file system across AZs.
- FinSight uses S3, because receipts are whole objects uploaded and read by URL.
- **Viva:** "EFS is shared network file storage; EBS is a single-instance disk; S3 is object storage."

Module 3 — Networking and Content Delivery

Amazon VPC [Syllabus]

- **Def:** a logically isolated virtual network in a Region, with your own IP range (CIDR).
- **Parts:**
- subnets;
- route tables;
- internet gateway;
- NAT gateway;
- security groups;
- network ACLs;
- VPC endpoints (gateway/interface);
- flow logs.
- **+** Network isolation and control.
- **–** Design effort; NAT gateways cost hourly plus per GB.
- **Eg:** a 3-tier app with public ALB subnets, private app subnets and private DB subnets.
- **FinSight:** the Lambdas run *outside* a VPC. They need no private resources, which avoids NAT cost and cold-start penalties.
- **Viva:** "A VPC is your private network in AWS; subnets split it across AZs."

Subnets

- A subnet is a CIDR slice of the VPC in **one AZ**. A **public** subnet has a route to an internet gateway; a **private** subnet has none (outbound only via NAT).
- **Viva:** "Public vs private is decided by the route table, not by the subnet's name."

Route tables

- **Def:** rules mapping destination CIDRs to targets (local, IGW, NAT, peering, TGW, endpoints). Each subnet is associated with one route table; the most specific route wins.
- **Viva:** "A route table tells subnet traffic where to go."

Security groups vs network ACLs

- **Security group:** instance/ENI-level, **stateful**, allow rules only, evaluated as a whole.
- **NACL:** subnet-level, **stateless** (return traffic must be allowed), allow and deny rules, evaluated in numbered order.
- **Viva:** "SGs are stateful firewalls on resources; NACLs are stateless firewalls on subnets."

VPC peering

- **Def:** a private connection between two VPCs (same or different account or Region).
- **–** Non-transitive; CIDRs must not overlap.
- **vs Transit Gateway:** a hub-and-spoke design for many VPCs and on-premises networks.
- **Viva:** "Peering links two VPCs privately but is not transitive."

AWS Direct Connect [Syllabus]

- **Def:** a dedicated private network connection from on-premises to AWS (1/10/100 Gbps, or hosted connections).
- **+** Consistent latency and bandwidth; lower egress rates.
- **–** Weeks to provision; not encrypted by default (add MACsec or VPN); one link is not highly available.
- **vs Site-to-Site VPN:** an encrypted tunnel over the internet, quick and cheap but variable.
- **Viva:** "Direct Connect is a private fibre link to AWS; VPN is an encrypted tunnel over the internet."

Amazon CloudFront [Planned]

- **Def:** a CDN that caches content at edge locations, with HTTPS, origin access control, Lambda@Edge / CloudFront Functions, and AWS WAF integration.
- **Parts:**
- distribution;
- origins (S3, ALB, API Gateway, custom);
- cache behaviours and policies;
- OAC;
- TLS certificates (ACM in us-east-1);
- invalidations.
- **+** Lower latency, less origin load, DDoS protection (Shield Standard), a free tier (1 TB/month).
- **–** Cache invalidation; configuration propagation takes minutes.

- **FinSight:** the template defines a private S3 bucket behind CloudFront with OAC, but creation was **refused because the new AWS account must be verified by AWS Support**. The SPA is served by a Lambda on API Gateway instead. CloudFront returns with `FrontendHosting=cloudfront` once enabled.
- **Viva:** "CloudFront caches content close to users; in FinSight it is planned but blocked by account verification, so it is not deployed."

Module 4 — Database Services

Amazon RDS [Syllabus]

- **Def:** managed relational databases: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Db2.
- **AWS handles** provisioning, patching, backups (point-in-time restore), Multi-AZ and read replicas.
- **+** SQL, joins, transactions, a familiar toolset.
- **−** Instance-based (pay while idle); vertical scaling; connection limits (mitigated by RDS Proxy).
- **Viva:** "RDS is managed SQL: AWS runs the engine and you design the schema and queries."

Multi-AZ deployments

- **Def:** a synchronous standby (or, in Multi-AZ DB clusters, 2 readable standbys) in another AZ, with automatic failover (typically 1–2 min) and the same endpoint.
- **Purpose: high availability**, not read scaling.
- **Viva:** "Multi-AZ is for failover; read replicas are for read scaling."

Read replicas

- **Def:** asynchronously replicated copies for read traffic (in-Region or cross-Region), which can be promoted.
- **−** Replication lag means eventual consistency.
- **Viva:** "Read replicas offload reads and can serve disaster recovery."

Amazon Aurora [Syllabus]

- **Def:** an AWS-built MySQL/PostgreSQL-compatible engine with a distributed storage layer (6 copies across 3 AZs), up to 15 low-lag replicas, fast failover, Aurora Serverless v2, and Global Database.
- **+** Higher performance and availability than standard RDS engines.
- **−** Higher price; AWS-specific.
- **Viva:** "Aurora is cloud-native MySQL/PostgreSQL with shared, self-healing storage across 3 AZs."

Amazon DynamoDB [FinSight]

- **Def:** a serverless key-value and document NoSQL database with single-digit-ms latency at any scale.
- **Parts:**
- tables;
- items;
- partition key (+ optional sort key);
- GSIs/LSIs;

- on-demand or provisioned capacity;
- TTL;
- transactions;
- Streams;
- PITR;
- global tables.
- **+** No servers, automatic scaling, pay-per-request, highly available across 3 AZs.
- **−** Access patterns must be designed up front; limited ad-hoc queries; 400 KB item limit.
- **Eg [FinSight]:** one on-demand table with `PK = USER#<sub>` and `SK = <TYPE>#<id>`, plus `GSI1` for expense date ranges. Every query is scoped to one user's partition (isolation by key design); there is no Scan.
- **Viva:** "DynamoDB is serverless NoSQL; you model the table around your queries."
- **vs RDS:**
- DynamoDB = no servers, no idle cost, scales automatically, no joins;
- RDS = SQL flexibility, instance cost.
- FinSight's access patterns are all per-user and known, so DynamoDB fits.

DynamoDB Streams [Syllabus]

- **Def:** a time-ordered log of item changes (kept 24 h), consumed by Lambda or Kinesis for triggers, auditing or replication.
- **Eg:** sending a notification when a budget item changes. Not used in FinSight.
- **Viva:** "Streams turn table changes into events, often processed by Lambda."

Module 5 — Management and Cost

Amazon CloudWatch [FinSight]

- **Def:** monitoring and observability.
- **Parts:**
- metrics (with namespaces and dimensions);
- Logs (log groups/streams, Insights queries, retention);
- alarms (with SNS/Auto Scaling actions);
- dashboards;
- Events (now EventBridge);
- Synthetics.
- **Eg [FinSight]:**
- 9 Lambda log groups with **14-day retention**;
- a structured request log (method, path, status, duration; no secrets);
- an alarm on API Gateway `5xx ≥ 5 / 5 min` (no SNS target yet).
- **Viva:** "CloudWatch collects metrics and logs and raises alarms."

AWS CloudTrail [Syllabus]

- **Def:** records AWS API calls (who, what, when, from where) as management/data events; 90 days of event history are free; trails deliver logs to S3.
- **vs CloudWatch:** CloudTrail = *who called which API*; CloudWatch = *how resources are performing*.
- **Viva:** "CloudTrail is the audit log of the AWS account."

AWS Config [Syllabus]

- **Def:** records resource configurations over time and evaluates them against rules (e.g. "S3 buckets must block public access"), with remediation.
- **Viva:** "Config answers what a resource looked like and whether it is compliant."

AWS Systems Manager [Syllabus]

- **Def:** operations hub.
- **Parts:**
- Session Manager (shell without SSH keys);
- Run Command;
- Patch Manager;
- Parameter Store (configuration and secrets);
- Automation;
- Inventory.
- **Viva:** "Systems Manager manages and patches fleets and stores parameters."

AWS Cost Management, Billing, Budgets [FinSight-relevant]

- **Parts:**
- Billing console (bills, credits, payment);
- Cost Explorer (analysis and forecasts; its API costs US$0.01 per request);
- AWS Budgets (alerts at thresholds; the first 2 budgets are free);
- Cost Anomaly Detection;
- cost allocation tags.
- **Key point: budgets alert; they do not stop spending.**
- **Eg [FinSight]:** Cost Explorer showed US$0.00 month to date. The account has a "Zero-Spend" budget; the estimate is ≈ US$0.05–0.10/month at demo scale.
- **Viva:** "Cost Explorer analyses spending, Budgets alert on thresholds, and tags attribute cost."

AWS Free Tier

- **Types:**
- always-free allowances (e.g. 1M Lambda requests/month, 25 GB DynamoDB storage, CloudFront 1 TB);
- short-term trials.
- **Accounts created after 15 July 2025** choose a **Free plan** (credits of up to US$200, part of them earned by completing activities, for up to 6 months, with no charges) or a **Paid plan**. On the Free plan the account closes when credits or time run out, unless upgraded.
- **FinSight:** the account is on the Free plan with US$159.91 credits until 2027-02-28. The credit balance rose by US$20 during the project, consistent with an earned-activity credit.

- **Viva:** "The Free Tier is a set of allowances and credits, not a guarantee that everything is free."

AWS Trusted Advisor [Syllabus]

- **Def:** checks across cost optimisation, performance, security, fault tolerance, service limits and operational excellence. Basic/Developer support gets core checks; Business and above get all.
- **Viva:** "Trusted Advisor gives best-practice recommendations for your account."

Module 6 — Security

Shared responsibility (security view)

- **AWS:** physical security, hypervisor, managed-service patching.
- **Customer:** IAM, data classification, encryption settings, network rules, application security.
- **FinSight examples:** the customer side covers Cognito configuration, JWT validation, revocation checks, input validation, the S3 public-access block and IAM least privilege.

AWS KMS [Syllabus]

- **Def:** a managed service for encryption keys (HSM-backed).
- **Parts:**
- AWS-owned, AWS-managed and customer-managed keys (CMKs);
- key policies;
- envelope encryption (data keys);
- rotation;
- CloudTrail audit of key use.
- **+** Central control and audit of encryption.
- **−** A per-key monthly cost and per-request cost for CMKs.
- **FinSight:** uses AWS-managed encryption (S3 SSE-S3/AES-256, DynamoDB default encryption) and **no customer-managed KMS keys**, to avoid cost.
- **Viva:** "KMS creates and controls keys; services use envelope encryption with data keys."

AWS WAF [Syllabus / future]

- **Def:** a Layer-7 web application firewall for CloudFront, ALB, API Gateway REST APIs, AppSync, Cognito and others.
- **Rules:** managed rule groups (SQLi, XSS, bad bots), rate-based rules, IP sets, geo-match.
- **FinSight:** not deployed. Planned with CloudFront (~US$5/month per web ACL plus rules).
- **Viva:** "WAF filters HTTP requests by rules, e.g. blocking SQL injection or rate-limiting an IP."

AWS Shield [Syllabus]

- **Shield Standard** (free, automatic) protects against common L3/L4 DDoS attacks. **Shield Advanced** (paid) adds L7 support with WAF, a response team (SRT) and cost protection.
- **Viva:** "Shield Standard is on by default; Advanced adds DDoS experts and cost protection."

IAM best practices

- Lock away the root user: MFA, no access keys, use it only for account tasks. *(FinSight: root MFA on and no keys — but deployments used root through `aws login`; an IAM Identity Center admin is recommended.)*
- Use federation or IAM Identity Center with temporary credentials.
- Grant least privilege and use roles for workloads.
- Rotate or avoid long-term keys; enable CloudTrail; use Access Analyzer; set permission boundaries and SCPs in organisations.
- **Viva:** "Never use root day-to-day; use roles and MFA, and grant the minimum."

Encryption

- **At rest:** SSE-S3, SSE-KMS, DynamoDB encryption, EBS encryption.
- **In transit:** TLS/HTTPS; S3 bucket policies can deny `aws:SecureTransport=false`.
- **Client-side** encryption is the customer's choice.
- **FinSight:** HTTPS only (execute-api), HSTS, TLS-only S3 policy, SSE-S3 for receipts, DynamoDB SSE.

Secrets handling

- **Services:**
- **AWS Secrets Manager** (rotation, ~US$0.40 per secret per month);
- **SSM Parameter Store** (SecureString, standard tier free).
- **Never put secrets in code or environment files committed to git.**
- **FinSight:**
- no application secrets are needed in AWS: Cognito issues tokens, verified with public JWKS, and Lambda uses IAM roles;
- pool and client ids are public identifiers;
- local test credentials live only in git-ignored files.
- **Viva:** "Store secrets in Secrets Manager or Parameter Store and grant read access through IAM."

Module 7 — Integration and Architecture

Amazon SQS [Syllabus]

- **Def:** a fully managed message queue for decoupling.
- **Types:** Standard (at-least-once delivery, best-effort order, nearly unlimited throughput) and FIFO (ordered, exactly-once processing, 300–3,000 msg/s with batching).
- **Parts:** visibility timeout, dead-letter queue, long polling, delay queues, retention up to 14 days.
- **Eg:** queue OCR jobs so uploads return immediately. *(FinSight processes OCR synchronously instead — a future option.)*
- **Viva:** "SQS buffers work between producers and consumers; consumers pull messages."

Amazon SNS [Syllabus]

- **Def:** pub/sub messaging. One topic fans out to many subscribers (SQS, Lambda, HTTP, email, SMS, mobile push).

- **vs SQS:** SNS *pushes* to many subscribers; SQS *queues* for pull-based consumers. A common pattern is SNS → multiple SQS queues (fan-out).
- **FinSight:** not used. Recommended for alarm email notifications.
- **Viva:** "SNS broadcasts; SQS queues."

Amazon SWF [Syllabus]

- **Def:** Simple Workflow Service coordinates multi-step workflows with deciders and activity workers that you host.
- **Status:** still available, but AWS recommends **Step Functions** for new workflows.
- **Viva:** "SWF is the older workflow service; Step Functions is its modern serverless successor for most cases."

AWS Step Functions [Syllabus]

- **Def:** serverless state machines (Amazon States Language) that orchestrate Lambda and 200+ AWS services.
- **Features:** retries, catch, parallel and map states.
- **Workflow types:** Standard (up to 1 year, exactly-once) and Express (high volume, ≤ 5 min).
- **Eg:** receipt pipeline upload → OCR → validate → notify.
- **Viva:** "Step Functions orchestrates services visually, with built-in retries and error handling."

AWS Well-Architected Framework

- **Six pillars:**
- **Operational excellence**;
- **Security**;
- **Reliability**;
- **Performance efficiency**;
- **Cost optimisation**;
- **Sustainability**.
- **Tools:** the Well-Architected Tool and Lenses (e.g. the Serverless Lens).
- **FinSight mapping:**
- IaC with SAM and changesets → operational excellence;
- least privilege, two-layer auth and private S3 → security;
- multi-AZ managed services and Retain on data → reliability;
- right-sized Lambda memory and on-demand DynamoDB → performance and cost;
- no idle servers → sustainability.
- **Viva:** "Six pillars to review an architecture against best practice."

Scalability vs elasticity

- **Scalability:** the ability to handle more load by adding resources (vertical = bigger, horizontal = more). **Elasticity:** adding *and removing* resources automatically as demand changes.
- **FinSight:** Lambda and DynamoDB on-demand are elastic automatically (API stage throttle 25 rps, burst 50).

- **Viva:** "Scalability is capacity to grow; elasticity is automatic grow-and-shrink."

High availability vs fault tolerance

- **HA:** minimal downtime (e.g. Multi-AZ failover within minutes). **Fault tolerance:** continues with *no* interruption despite component failure (redundant active components).
- **FinSight:** Regional serverless services run across AZs. There is no multi-Region DR (out of scope).
- **Viva:** "HA reduces downtime; fault tolerance tolerates failures without any."

Security and cost optimisation as architecture principles

- **Security:** defence in depth, least privilege, encryption, automate and trace.
- **Cost:** pay for value; serverless and on-demand; right-size; lifecycle rules; tags; delete unused resources (e.g. old ECR images); budgets.

Module 8 — Migration and Hybrid Architecture

AWS DMS (Database Migration Service) [Syllabus]

- **Def:** migrates databases with minimal downtime (full load + CDC). Supports homogeneous (Oracle → RDS Oracle) and heterogeneous (Oracle → Aurora PostgreSQL, with AWS SCT / DMS Schema Conversion) migrations. DMS Serverless is also available.
- **Viva:** "DMS copies data and keeps replicating changes until cut-over."

AWS SMS (Server Migration Service) [Syllabus — discontinued]

- **Def:** agentless migration of on-premises VMware/Hyper-V/Azure VMs to AMIs by replicating snapshots.
- **Status: AWS ended SMS (no new use since 2022). The replacement is AWS Application Migration Service (MGN)**, which does continuous block-level replication with minimal cut-over downtime.
- **Viva:** "SMS migrated VMs incrementally; it has been replaced by Application Migration Service."

AWS Snow Family [Syllabus]

- **Snowball / Snowball Edge:**
- rugged devices shipped to you for offline transfer of terabytes to petabytes;
- Snowball Edge adds edge compute (EC2-compatible instances, Lambda);
- Storage Optimized and Compute Optimized variants;
- data is encrypted (KMS) and the device is tamper-evident.
- **Snowmobile:** a 45-foot shipping-container data centre for exabyte-scale transfers. **AWS has retired Snowmobile.** Large transfers now use multiple Snowball Edge devices or online options.
- **Note:** AWS has also changed the Snowball device line-up over time (older Snowball models retired). Check the current Snow Family page before quoting device sizes.
- **When to use:** when network transfer would take too long or cost too much (rule of thumb: > ~10 TB over a slow link), or for edge computing in disconnected sites.
- **vs online transfer:** DataSync and Transfer Family move data over the network; Direct Connect gives a private link.
- **Viva:** "Snow devices move bulk data physically; Snowball Edge can also compute at the edge."

Hybrid architecture (summary)

- **Connectivity:** Site-to-Site VPN, Direct Connect, Transit Gateway.
- **Storage:** Storage Gateway (file/volume/tape).
- **Compute:** Outposts (AWS hardware on-premises), Local Zones.
- **Identity:** IAM Identity Center with an on-premises AD.
- **Viva:** "Hybrid connects on-premises and AWS so workloads and data can span both."

Quick comparisons (frequent viva pairs)

Pair	One-line answer
EC2 vs Lambda	servers you run and pay for while up vs functions run per event and billed per ms (FinSight: Lambda)
S3 vs EBS vs EFS	objects over HTTPS vs single-instance block disk in one AZ vs shared NFS across AZs
DynamoDB vs RDS	serverless NoSQL with designed access patterns vs managed SQL with joins and an instance cost
Multi-AZ vs read replica	synchronous standby for failover vs asynchronous copy for reads
SG vs NACL	stateful, allow-only, resource level vs stateless, allow/deny, subnet level
SQS vs SNS	pull queue vs push fan-out
CloudWatch vs CloudTrail	performance metrics/logs vs API audit trail
ALB vs API Gateway	L7 load balancer with an hourly cost vs managed API front door with auth/throttling and per-request cost
KMS vs Secrets Manager	manages encryption keys vs stores and rotates secrets (encrypted with KMS)
Step Functions vs SWF	serverless state machines (recommended) vs older worker-hosted workflows

---

Part II — FinSight project viva (150 questions)

A. Project overview

1. What is FinSight? A cloud-based personal finance platform that combines: - expense tracking; - budgets; - analytics; - an explainable insight engine; - anomaly review; - receipt OCR; - a What-If simulator; - savings goals; - a rule-based assistant.

It runs on AWS serverless services in Mumbai (ap-south-1).

2. What is the full title? FinSight — Cloud-Based Personal Finance Intelligence & Expense Analytics Platform.

3. What is the live URL? https://zkht2b4ib3.execute-api.ap-south-1.amazonaws.com. It is an API Gateway URL that serves both the web app and the API.

4. What makes FinSight different from a simple expense tracker? Every insight is explained. It is labelled as fact, estimate, projection or recommendation, and shows the period, the calculation, the values and the transactions behind it. Anomalies are flagged for review, never auto-changed. Receipts are scanned but always confirmed by the user.

5. Which features are deliberately not supported? - income tracking; - bank connections; - investments; - multi-currency conversion.

Each account has one currency, and the UI never invents these features.

6. Is any paid AI used? No. Insights, anomaly detection and the assistant are deterministic rules and statistics. OCR is open-source Tesseract running in our own Lambda.

B. Problem statement and objectives

7. What problem does FinSight solve? People's spending is scattered and unexplained, receipts are typed in by hand, and overspending or unusual charges are noticed too late.

8. List the objectives. 1. Record and organise expenses. 2. Monitor budgets with early warnings. 3. Visualise trends. 4. Explain insights with evidence. 5. Flag unusual transactions. 6. Digitise receipts. 7. Model savings scenarios. 8. Track goals. 9. Deploy securely and cheaply on AWS.

9. Who is the target user? An individual (for example a student or a young professional) managing personal spending in one currency, defaulting to INR and the Asia/Kolkata timezone.

10. How do you measure success? - functional correctness: 131 backend and 20 frontend automated tests; - live end-to-end verification on AWS: 64/64 API checks and 27/27 browser checks; - security properties: isolation, private storage and revocation, all tested live; - low running cost: about US$0.05–0.10/month estimated at demo scale.

C. Frontend architecture

11. Which frontend technologies are used? - React 19 and TypeScript, built with Vite 8; - Tailwind CSS v4; - React Router 8; - Recharts 3 for charts; - React Hook Form + Zod for forms; - Radix Dialog; - Sonner toasts; - Lucide icons; - self-hosted Inter font.

12. How is the frontend organised? - `pages/` holds one file per route. - `components/ui/primitives.tsx` holds shared buttons, cards, inputs and progress bars. - `components/charts/` holds the charts. - `lib/api.ts` is the single API client. - `lib/auth.tsx` handles local or Cognito authentication. - `lib/format.ts` handles money stored as integer minor units.

13. How does the frontend know whether to use Cognito? It calls the public `GET /api/v1/config` endpoint, which returns `auth_mode`, the Cognito pool id and client id, and the upload mode.

14. Where are tokens stored? In `sessionStorage`, which is cleared when the tab closes. The access token is sent as `Authorization: Bearer`.

15. How are themes handled without a flash of the wrong theme? An inline script in `index.html` sets the `dark` class before the first paint. Dark is the default. Switching themes adds a short colour cross-fade, which is skipped when the user prefers reduced motion.

16. What animations were added and how are they made accessible? They are CSS-first, with no new library: - page stagger; - the sidebar indicator; - the drawer slide; - counting numbers; - chart draw-in on entry and data change only; - growing progress bars; - receipt processing steps; - message entry.

`prefers-reduced-motion` disables all of them. Counting numbers expose only the final value to screen readers.

17. What is the About the author feature? An "i" button in the landing header and a "Created by Aakar Gupta" credit in the footer open an accessible Radix dialog. It has focus trapping, Escape to close and focus return. It shows the role, the institution and a verified technology list, which states that CloudFront is planned and not yet enabled.

18. Why is money stored as integers? Floating point cannot represent paise exactly. All amounts are integer minor units (₹450.50 is stored as 45050), and the fields end in `_minor`.

D. Backend architecture

19. Which backend stack is used? - Python 3.13 on Lambda (3.14 locally); - FastAPI; - Pydantic v2; - Mangum, which adapts API Gateway events to ASGI; - PyJWT; - Pillow and pytesseract.

20. Describe the layers. - `api/routes` handles HTTP and validation. - `services` holds the business logic: budgets, analytics, insights, anomalies, simulator, goals, assistant and receipts. - `repositories` handles persistence: SQLite locally and DynamoDB on AWS, behind one `Repository` interface.

21. How does one codebase run locally and on Lambda? `create_app()` builds the FastAPI app. `app/handlers/<group>.py` wraps a subset of routes with Mangum, one Lambda per route group. Locally, uvicorn serves all routes with SQLite and local file storage.

22. Why split into several Lambda functions instead of one? - smaller blast radius; - per-function IAM, so only the receipts function can read and write S3; - different memory sizes: OCR uses 1024 MB, analytics 768 MB; - independent logs and metrics.

23. How are errors returned? As one envelope: `{"error": {"code", "message", "details"}}`. Validation errors list each field. A 500 error never exposes internals. In production the OpenAPI docs are disabled.

24. How are unknown fields handled? Request models are strict, so an unknown field such as `user_id` is rejected with 422. This was tested live as SEC-10.

E. API design

25. What is the base path? `/api/v1`. Resources include `expenses`, `budgets`, `dashboard`, `analytics`, `insights`, `anomalies`, `receipts`, `goals`, `simulations`, `assistant/query`, `me`, `preferences` and `config`.

26. How does pagination and filtering work for expenses? `GET /expenses` accepts: - `start` and `end`; - `category` and `payment_method` (both repeatable); - `search`; - `min_amount_minor` and `max_amount_minor`; - `sort` and `order`; - `page` and `page_size` (up to 100).

The response totals cover all matching rows, not just the current page.

27. Which endpoints are public? `GET /api/v1/health`, `GET /api/v1/config`, and the static web routes (`GET /`, `GET /{proxy+}`). Everything else needs a valid token.

28. How do you prevent the static route from hiding API errors? API Gateway always prefers the more specific `/api/v1/...` routes. The web function also returns a JSON 404 for any unmatched `/api/*` path instead of the SPA.

F. Authentication and authorization

29. How does sign-up work? 1. The SPA calls Cognito `SignUp` with email, password and name. 2. Cognito emails a confirmation code. 3. The user enters the code and the SPA calls `ConfirmSignUp`.

30. How does sign-in work? Cognito `InitiateAuth` with `USER_PASSWORD_AUTH` on a public app client with no secret. It returns ID, access and refresh tokens. Access and ID tokens last 60 minutes; refresh tokens last 7 days.

31. Which token is sent to the API, and why? The **access** token. It is meant for authorising API calls and carries `client_id` and `token_use=access`. The ID token is only used by the UI to show the name and email.

32. Describe the two layers of authorization. 1. **The API Gateway JWT authorizer** checks the signature, issuer (the user pool), audience (the app client) and expiry before any Lambda runs. 2. **The backend** re-verifies the RS256 signature with the pool's JWKS, plus issuer, expiry, `token_use` and `client_id`. It then checks revocation with Cognito `GetUser`.

33. Where does the backend get the user id? Only from the verified token's `sub` claim, never from the body, the query string or the path.

34. What happens on sign-out? 1. The SPA calls `GlobalSignOut` and clears session storage. 2. Cognito revokes the tokens. 3. The backend's `GetUser` check rejects the old access token with **401** `token_revoked`. The live test confirmed 200 before sign-out and 401 about 17 s after.

35. Why not just rely on the API Gateway authorizer? It validates only the signature and expiry, so a token revoked by sign-out would keep working until it expired (up to 60 min). Our first live run showed exactly this (AUTH-07). We fixed it with the backend revocation check.

36. Doesn't calling Cognito on every request add latency? The result is cached for 15 s per token in each warm container, so most requests don't call Cognito. The trade-off is that revocation takes effect within 15 s rather than instantly.

37. What if Cognito is throttled or down? The check **fails open** and logs a warning. The token is still cryptographically valid, and this avoids locking every user out because a dependency failed. It is documented as a known trade-off.

38. How is local (non-AWS) authentication done? - scrypt password hashes; - HS256 JWTs signed with a generated secret; - login rate limiting (5 failures per 5 min).

Production AWS uses Cognito instead.

39. Can one user read another user's data? No. Every DynamoDB key is prefixed with `USER#<sub>`, and every query uses the caller's partition. Live test ISO-01 made 6 cross-user requests (read, update, delete, receipt, image, goal) and all returned 404.

40. Why return 404 instead of 403 for other users' records? 404 doesn't reveal that the record exists.

G. DynamoDB data model

41. Why one table? A single-table design keeps all of a user's items in one partition (`PK = USER#<sub>`), with the item type in the sort key (`EXPENSE#…`, `BUDGET#…`, `GOAL#…`, `RECEIPT#…`, `PREFS` …). One table also means one IAM policy and no joins.

42. What is GSI1 for? Querying expenses by date range per user. `GSI1PK` and `GSI1SK` encode the user and the date, so a month's expenses are a single `Query`.

43. Do you use Scan? No. Every access is `GetItem`, `Query`, `PutItem`, `UpdateItem`, `DeleteItem`, `BatchWriteItem` or a transaction. The IAM policy does not even allow Scan.

44. Which capacity mode and why? On-demand (PAY_PER_REQUEST). It has no idle cost, needs no capacity planning, and suits unpredictable demo traffic.

45. What happens to the table if the stack is deleted? It is kept, because `DeletionPolicy: Retain` protects the data. It has to be deleted manually.

46. Is point-in-time recovery on? No; it was left off for cost. Turning it on is recommended for real users.

47. How are transactions used? `transact_write_items` with condition expressions keeps two writes atomic: - creating a local user together with a unique-email marker item; - creating an anomaly alert together with a de-duplication key (`ANOMALYKEY#…`, `attribute_not_exists`), so the same alert is never stored twice.

IAM allows these through the underlying Put and ConditionCheck actions.

H. S3 receipt storage

48. How does a receipt get into S3? 1. The backend issues a **presigned POST** that is valid for 5 minutes. Its policy fixes the key (`<sub>/<uuid>.<ext>`), the content type, AES256 server-side encryption and the size (1 byte to 5 MB). 2. The browser uploads the file directly to the private bucket. The file never passes through API Gateway.

49. How is a receipt shown later? `GET /receipts/{id}/image` returns a 2-minute presigned GET URL, which the browser uses as the image source.

50. Is the bucket public? No. All four Block Public Access settings are on, and a bucket policy denies non-TLS requests. An unsigned GET returns 403 (live test RCT-08).

51. What stops an oversized upload? The presigned POST policy (`content-length-range`). S3 itself rejected a 6 MB upload with `EntityTooLarge` (RCT-05).

52. Why did presigned uploads first return 307? boto3 signed the request for S3's global endpoint. A newly created bucket outside us-east-1 answers the global endpoint with temporary redirects, which break browser POST uploads. We pinned the client to the regional endpoint (`https://<bucket>.s3.ap-south-1.amazonaws.com`) and added a regression test.

53. Why does S3 CORS allow `*.execute-api.ap-south-1.amazonaws.com`? In the current hosting mode the site's origin is the API's own execute-api URL. Referencing it exactly in the bucket would create a circular template dependency. Writing still requires a valid presigned policy for a server-generated key. With CloudFront the template uses the exact origin.

I. OCR and Tesseract

54. Which OCR engine is used? Tesseract 5.5.0 (eng) inside a container-image Lambda (`ReceiptsFunction`, 1024 MB, 60 s timeout).

55. Why a container instead of a zip package? Tesseract is a native binary with language data. A Debian-based image (`python:3.13-slim` + `tesseract-ocr` + `awslambdaric`) installs it reliably, and images can be up to 10 GB.

56. What preprocessing is applied? 1. EXIF rotation. 2. Grayscale. 3. Resize to between 1000 and 2500 px width (Tesseract prefers about 300 DPI text). 4. Auto-contrast. 5. Sharpen.

57. Which Tesseract mode? `--oem 3 --psm 4`: the default LSTM engine, with the page treated as a single column of text of variable sizes, which suits receipts.

58. How are fields extracted? A rule-based parser finds the merchant, the date (several formats), the total, the tax, the subtotal, the currency and line items. Each field has a confidence and a source. The total is cross-checked against subtotal + tax when both exist.

59. What did the live OCR test extract? From the synthetic receipt: merchant "FRESH MART SUPERMARKET", date 2026-10-14, total 610.05, tax 29.05, with 93.3% mean OCR confidence, in about 2–3 s on Lambda.

60. Does OCR create the expense automatically? No. The user reviews and corrects every field, then confirms. Only then is a linked expense created (RCT-09).

61. What validation happens before OCR? - type allow-list (JPEG, PNG, WebP); - magic-byte sniffing (the declared type must match); - size limit; - decodability check; - a 30 s OCR timeout.

PDFs are rejected with 422 (RCT-10).

J. AWS Lambda

62. How many functions are deployed? Nine: - seven zip functions on python3.13 (expenses, budgets, analytics, anomalies, planning, assistant, account); - one container function (receipts/OCR); - one static web function (WebFunction).

63. Memory and timeouts? - 512 MB / 15 s by default; - analytics 768 MB; - anomalies and account 30 s; - OCR 1024 MB / 60 s; - web 256 MB / 10 s.

64. How were the zip packages built for Python 3.13 on a Windows machine running Python 3.14? `sam build` with aws-lambda-builders downloads **manylinux x86_64 cp313 wheels** from `requirements.txt`. A local Python 3.13 is needed only for SAM's runtime check, so no Windows binaries end up in the package. We verified the imports in a Linux cp313 container before deploying.

65. What packaging bug did you hit? At first the wheels were vendored into the source folder, but SAM's Python builder skips `*.so` files when it copies source. The `pydantic_core` native module disappeared and `/health` returned 500 (`ImportModuleError`). Staging `requirements.txt` instead fixed it.

66. How fast are the functions? Warm requests take 15–130 ms. Cold starts take about 3 s per zip function at 512 MB, and the first OCR container start takes about 15 s.

67. Why not use provisioned concurrency? It costs money even while idle. Demo traffic doesn't justify it.

68. Are the Lambdas in a VPC? No. They need no private network resources, so we avoid NAT gateway cost and VPC cold-start overhead.

K. API Gateway

69. REST API or HTTP API, and why? HTTP API. It is cheaper (about US$1 per million requests), has lower latency, and has a native JWT authorizer for Cognito.

70. What throttling is configured? A rate of 25 requests/s and a burst of 50 on the default route settings.

71. How is CORS handled? The SPA is served from the same execute-api origin, so browser API calls are same-origin and need no CORS. The HTTP API CORS configuration and the backend allow-list contain only a placeholder or the configured origins (never `*`). A preflight from a foreign origin gets no `Access-Control-Allow-Origin` header (COR-02).

72. Why does API Gateway also serve the frontend? CloudFront creation was blocked for this new account, so a small stdlib Lambda (`WebFunction`) serves the built SPA: - `index.html` with `no-cache`; - hashed assets with a one-year `immutable` cache; - deep links fall back to `index.html`; - security headers (HSTS, nosniff, X-Frame-Options, Referrer-Policy).

L. Amazon Cognito

73. Which Cognito tier? The **Lite** feature plan, which is enough for email sign-in and has the lowest cost (10,000 MAU free).

74. Key user-pool settings? - email as the username; - email auto-verified; - password at least 8 characters with numbers; - recovery by verified email; - Cognito's default email sender.

75. App client settings? - no client secret (public SPA); - `ALLOW_USER_PASSWORD_AUTH` and `ALLOW_REFRESH_TOKEN_AUTH`; - PreventUserExistenceErrors on; - token revocation enabled; - access/ID tokens 60 min, refresh 7 days.

76. Why does the profile not show a UUID as the name? Cognito access tokens carry no name, and with email sign-in the `username` claim is a UUID. We fixed a bug where that UUID became the display name. The backend now leaves the name empty, and the UI uses the ID token's name.

77. How were test users confirmed without an email inbox? They were created with the public `SignUp` call (the same call the web app makes) and confirmed with `AdminConfirmSignUp` through the CLI. Real users confirm with the emailed code.

M. CloudWatch

78. What is logged? One line per request: method, path, status, duration and request id. No bodies, tokens or query strings are logged. Lambda also writes its `REPORT` lines (duration, memory).

79. Log retention? 14 days, set explicitly on all 9 log groups so that logs don't accumulate forever.

80. Which alarm exists? `ApiServerErrorsAlarm`, which fires when API Gateway `5xx` is at least 5 in 5 minutes. Its state is OK. It has no SNS notification target yet; we recommend adding one.

81. How did monitoring help during deployment? The logs showed the `ImportModuleError` immediately after the first deployment, which led straight to the packaging fix.

N. IAM

82. What permissions do the API functions have? `AWSLambdaBasicExecutionRole` (logs) plus a shared managed policy that allows GetItem, PutItem, UpdateItem, DeleteItem, Query, BatchWriteItem, ConditionCheckItem and DescribeTable on **one table and its indexes**.

83. Which functions can access S3? Only `ReceiptsFunction` (Get, Put and Delete on `receipts-bucket/*`) and `AccountFunction` (Delete only, used for data deletion). WebFunction has logs only.

84. Does `GetUser` need IAM permission? No. It is authorised by the user's access token.

85. Which identity deployed the stack? The root user, through `aws login` (short-lived console credentials) with MFA enabled and no access keys. We recommend an IAM Identity Center admin user for future deployments.

O. SAM and CloudFormation

86. Why AWS SAM? It defines Lambda functions, the HTTP API, events and IAM with short syntax, builds both zip and container functions, and deploys through CloudFormation changesets.

87. What is a changeset and how did you use it? A preview of the resource changes. We ran `sam deploy --no-execute-changeset`, reviewed it (e.g. 63 adds and 0 deletes at first creation; later updates were code-only with Replacement False), then executed it.

88. How many stacks exist? Three: - `finance-intelligence-prod` (63 resources); - the SAM companion stack for the ECR repository; - `aws-sam-cli-managed-default` (the artifact bucket).

89. What does the `FrontendHosting` parameter do? - `cloudfront` (the default) creates the web bucket, OAC and distribution. - `apigateway` (deployed now) skips them, and the SPA is served by WebFunction.

CORS rules and outputs change with it.

90. What happens on a failed stack creation? CloudFormation rolls back. Retained resources (the table) survive, so after the CloudFront failures we deleted the empty, just-created table (with owner approval) before retrying.

P. Deployment process

91. Walk through a deployment. 1. `python infra/prepare_build.py` stages the backend and `requirements.txt`, and builds the same-origin SPA into `infra/build/web`. 2. `sam build` resolves Linux wheels and builds the OCR Docker image. 3. `sam deploy` reviews the changeset (profile `finsight`, ap-south-1) and executes it. 4. Run `infra/live_test.py` and `docs/evidence/capture/live.mjs`.

92. How was the wrong AWS account avoided? The first sign-in reached an old account in `PENDING_ACTIVATION` where every service was blocked. We confirmed with the owner, signed out, and used a dedicated CLI profile `finsight` pinned in `samconfig.toml`. The account id is checked before every change.

93. What deployment problems did you face? 1. CloudFront was blocked by account verification. 2. SAM dropped `*.so` files from vendored wheels. 3. S3 returned 307 for presigned POST. 4. Sign-out did not revoke access tokens.

All four were diagnosed from logs or tests and fixed or worked around.

94. How is the frontend deployed in CloudFront mode? `infra/deploy_frontend.py` reads the stack outputs, builds with `VITE_API_BASE_URL=ApiUrl`, syncs `dist/` to S3 (hashed assets immutable, `index.html` no-cache) and creates a CloudFront invalidation.

Q. Security

95. List the main security controls. - HTTPS only; - two-layer token validation plus revocation; - user id only from the token; - per-user partition keys; - strict input validation; - least-privilege IAM; - private S3 with presigned URLs, SSE and TLS-only access; - no secrets in code; - metadata-only logging; - throttling; - 0 known dependency vulnerabilities (npm audit, pip-audit).

96. What are the remaining security gaps? - deployment uses the root user; - no Content-Security-Policy header; - no WAF (needs CloudFront or an ALB); - the alarm has no notifications; - PITR is off.

Each has a recommendation in `docs/PROJECT_DOCUMENTATION.md` §16.4.

97. How do you protect against XSS? React escapes output and no user HTML is rendered. A CSP header is the recommended next step, because tokens live in `sessionStorage`.

98. How do you protect against injection? DynamoDB uses structured key conditions with no query strings to inject into. Inputs are validated by Pydantic. Receipt keys are generated by the server and checked against a regex.

99. Is the receipt data encrypted? Yes: SSE-S3 (AES-256) at rest, and in transit only over TLS. DynamoDB uses default encryption at rest.

R. Testing strategy

100. What tests exist? - **Backend: 131 pytest tests** covering money and dates, expenses, budgets, insights, anomalies, simulator and goals, the assistant, receipts, security, Cognito tokens (incl. revocation), DynamoDB and S3 via moto mocks, Lambda handlers and the static web handler. - **Frontend: 20 Vitest tests** covering formatting, the API client, motion helpers and the About dialog.

101. What is tested live on AWS? `infra/live_test.py` runs 64 checks: - frontend delivery; - public endpoints; - 401s for missing or forged tokens; - Cognito sign-up, sign-in, wrong password and refresh; - profile; - expense CRUD, search, filter, validation and 404; - budgets; - dashboard, analytics, insights and anomalies; - assistant, goals and simulator; - the full receipt flow including OCR; - isolation; - CORS; - revocation; - cleanup.

102. What is tested in a real browser? `live.mjs` runs headless Chrome against the live URL (27 checks): - Cognito sign-in through the UI; - creating an expense in the UI; - browser-to-S3 upload and OCR; - assistant and goals; - theme persistence; - the About dialog (dark, light and mobile); - no horizontal overflow at 390 px; - sign-out and protected-route redirect; - 0 console errors, 0 failed requests and HTTPS only.

103. How do you test AWS code without AWS? moto mocks DynamoDB and S3, and the Cognito tests use a locally generated RSA key standing in for the JWKS.

104. What did live testing find that local tests did not? The `*.so` packaging problem, the S3 307 redirect, and the access-token revocation gap.

105. How is test data handled? Only synthetic users (`@example.com`) and synthetic receipts are used. Isolation user B is deleted after each run, and earlier test users are purged at the start. The owner's real account is never touched.

S. Cost optimisation

106. What does FinSight cost? Cost Explorer showed US$0.00 month to date (its data lags up to 24 h). The estimate at demo scale (≤ 5k requests/month) is about **US$0.05–0.10/month**, mostly ECR image storage, charged against the Free-plan credits (US$159.91 remaining, plan until 2027-02-28). It is not guaranteed free: cost depends on usage.

107. Which choices keep it cheap? - serverless and on-demand only; - no VPC/NAT, EC2, RDS or load balancer; - HTTP API instead of a REST API; - Cognito Lite; - 14-day log retention; - PITR off; - no KMS

customer-managed keys; - no paid AI.

108. What still costs money while idle? ECR storage (three images, about 0.38 GB), plus KB-scale DynamoDB/S3 storage. Deleting the two superseded images would save about US$0.025/month. That needs owner approval.

109. Do AWS Budgets stop charges? No. They only send alerts. On the Free plan, AWS doesn't bill beyond the credits; instead the account closes when the credits or the time run out.

T. CloudFront limitation

110. Is CloudFront deployed? No. It is in the template (`FrontendHosting=cloudfront`), but creation failed twice with "Your account must be verified before you can add new CloudFront resources."

111. What is needed to enable it? An AWS Support case (Account and billing) asking AWS to verify the account for CloudFront. After that, set `FrontendHosting=cloudfront`, run `sam deploy` and `deploy_frontend.py`, and re-run the tests. The URL will change to a `*.cloudfront.net` domain.

112. What do you lose without CloudFront? Edge caching and lower latency outside Mumbai, CDN-level DDoS and WAF integration, and fewer Lambda invocations for static files. HTTPS and functionality are unchanged.

113. Why not use S3 static website hosting instead? S3 website endpoints are HTTP-only and need a public bucket, which is weaker than serving through API Gateway over HTTPS.

U. Token revocation limitation

114. Is a token revoked immediately on sign-out? Not instantly. It is rejected within about 15 s (the cache window), and the live test confirmed 401 about 17 s after `GlobalSignOut`. If Cognito is unreachable, the check is skipped and the request is allowed until the token expires (≤ 60 min).

115. How could revocation be made stricter? - set the cache to 0 (one Cognito call per request); - fail closed; - shorten the access-token lifetime; - use the Cognito Essentials tier with refresh-token rotation. Each trades latency, availability or cost.

V. Alternative architecture choices

116. Why not run FastAPI on EC2 or ECS? Always-on instances or tasks cost money while idle and need patching, scaling and a load balancer. Lambda costs about nothing while idle and scales automatically.

117. Why not Amplify Hosting? It would add another service and its own build pipeline. The SAM template already owns every resource, and Amplify doesn't address the account's CloudFront verification issue in a way we could verify.

118. Why not Textract for receipts? Textract is a paid, per-page service. Tesseract is free, open source and runs in our Lambda, which fits the no-paid-AI objective. Textract would likely be more accurate on poor photos.

119. Why not Step Functions for the receipt pipeline? Processing is one synchronous step of 2–3 s, so an orchestrator adds cost and complexity without benefit. It would make sense if OCR became asynchronous or multi-stage.

W. Database comparisons

120. Why DynamoDB instead of RDS? - all access patterns are per-user and known in advance; - there is no need for joins; - on-demand billing means no idle cost; - no instance to patch; - the data is automatically spread over 3 AZs.

RDS would cost hourly even at zero traffic.

121. When would RDS or Aurora be better? For ad-hoc SQL reporting across users, complex joins, or strong relational constraints.

122. Why SQLite locally? Zero setup for development and tests. The repository interface lets both backends share the same business code.

X. Serverless versus EC2

123. Give three advantages of serverless here. 1. No servers to patch. 2. It scales per request. 3. It costs almost nothing while idle.

124. Give three disadvantages. 1. Cold starts (about 3 s). 2. Execution limits (15 min). 3. Vendor-specific packaging, such as native wheels and container images.

Y. S3 versus EBS versus EFS

125. Why S3 for receipts? Receipts are whole objects written once and read by URL. S3 gives durability, private access with presigned URLs, direct browser upload and the lowest cost. EBS attaches to one EC2 instance, and EFS is a shared file system — neither fits browser uploads from Lambda.

Z. DynamoDB versus RDS (short)

126. One-line answer? "DynamoDB is serverless NoSQL designed around known queries; RDS is managed SQL for flexible queries with an instance cost."

AA. High availability and scalability

127. How available is FinSight? Lambda, API Gateway, DynamoDB, S3 and Cognito are Regional services that run across multiple AZs in ap-south-1. There is no multi-Region failover, which is out of scope.

128. How does it scale? Lambda adds concurrent executions per request, and DynamoDB on-demand scales throughput. The API stage throttle (25 rps / burst 50) protects the account and is the first limit to raise.

AB. Difficult project-specific questions

129. Your anomaly detector says a charge is unusual. Is it fraud? No. It means the charge *deserves review*. We use a modified z-score, $0.6745 \cdot (x - \text{median})/\text{MAD}$, per category, with presets of robust $z \geq 5.0$, 3.5 (default) or 3.0, plus ratio tests, duplicates within one day and monthly spikes. Transactions are never changed automatically.

130. Why median and MAD rather than mean and standard deviation? Spending is skewed, and a single large purchase inflates the mean and standard deviation, hiding outliers. Median and MAD are robust to this, especially with small samples (at least 5 per category).

131. How is the month-end projection computed? Spent so far ÷ days elapsed × days in the month. It is labelled a *projection*, not a certainty.

132. How does the What-If simulator compute savings? From a baseline (average of the last 3 months, last month, or the current-month projection), it applies per-category cuts (percent, amount or target). It reports monthly and annual savings, budget impact and months to the goal. Up to 4 scenarios are supported.

133. What exactly does the assistant do? It is rule-based natural-language matching to intents: total spent, count, average, top category, category change, budget remaining/status, unusual and largest transactions,

list, simulate, goal requirement, and help. It computes every answer from the user's data and shows the period. It is not generative AI.

134. What prevents the static web function from serving files outside its folder? Paths are resolved and must stay inside `static/`. Traversal attempts return 404; a unit test covers this.

135. If a Lambda reads a receipt for user A, can it be tricked into user B's file? No. The key comes from the receipt record, which is looked up in the caller's own partition, and keys are validated against a server-side pattern.

AC. Rapid-fire

136. Region? → ap-south-1 (Mumbai). **137.** Stack name? → `finance-intelligence-prod`. **138.** Database? → DynamoDB on-demand, single table + GSI1. **139.** Auth? → Cognito Lite + API Gateway JWT authorizer + backend JWKS and revocation check. **140.** OCR? → Tesseract 5.5 in a container Lambda. **141.** IaC? → AWS SAM / CloudFormation. **142.** Log retention? → 14 days. **143.** Upload limit? → 5 MB, JPEG/PNG/WebP. **144.** Presigned POST lifetime? → 5 min (GET: 2 min). **145.** CloudFront? → Planned, blocked by account verification, not deployed. **146.** Live test result? → 64/64 API, 27/27 browser. **147.** Money format? → integer minor units.

AD. Mock viva (short answers)

148. Draw the request path for "add expense". 1. The SPA sends `POST /api/v1/expenses` with the Bearer token to API Gateway. 2. The JWT authorizer passes it on, and ExpensesFunction (Mangum → FastAPI) runs. 3. The backend verifies the token and its revocation, then validates the body. 4. It writes `USER#<sub>` / `EXPENSE#<id>` to DynamoDB, re-runs anomaly and budget checks, and returns 201 with any new alerts.

149. What would you do next with one more week? 1. Get CloudFront enabled and add WAF. 2. Add a CSP header and SNS alarm emails. 3. Switch deployments to IAM Identity Center. 4. Add an ECR lifecycle policy. 5. Consider asynchronous OCR with SQS.

150. What was the most important lesson? Live testing on real AWS found problems that mocks could not: packaging, an S3 redirect and token revocation. Each fix was then turned into an automated test.